Project Report
Signed Arithmetic Pipeline: Design and Verification

Reece M. Dobro

May 25, 2026

Abstract

This report presents the design and verification of a program written in Verilog through AMD Vivado that performs signed arithmetic on four 8-bit two's complement inputs to produce a 10-bit output. The system stores inputs in registers A, B, C, and D and computes the expression (A-B)+(C-D). A modular design consisting of controller and datapath units was implemented using dataflow and gate-level modeling. Verification was conducted through targeted simulation test cases, including nominal operation, negative inputs, capture logic behavior, and edge conditions. Simulation results, evaluated through waveforms and console outputs, confirm that the system operates correctly, asserts the valid signal appropriately, and handles all tested conditions without overflow. The design satisfies all specified functional requirements.

1 Introduction, Background, and Theory

The main objective of this project is to create, build, and test a design that takes in 4, 8-bit 2's complement signed numbers through a single 8-bit 2's complement input value, then perform an arithmetic calculation with the 4 stored numbers. This calculation results in a 2's complement result, which is the main output of the design. The 4, 8-bit values are selected using a 2-bit value, *op*, and will be referred to in this paper as registers A, B, C, and D. The arithmetic calculation is (A-B)+(C-D). This project requires the use of only dataflow and gate-level modeling, except for the use of a D Flip-Flop module provided by design specifications. The external behavior of the design was given in its entirety through project specifications, and the internal behavior was partly given through project specifications. The testing of this design was done with 4 sets of test vectors. If the outputs of the design match the expected outputs, the design is deemed successful.

1.1 Top-Level Inputs and Outputs

The inputs, outputs, and expected behavior were provided by the project specifications. The top-level module, *project2*, contains all of the inputs and outputs. These inputs and outputs are shown in Table 1.

Signal	Direction	Size	Description
reset_n	Input	1	Active-low synchronous reset
clock	Input	1	Free-running clock
d_in	Input	8	2's complement input value
op	Input	2	Selector bit for A, B, C, D
capture	Input	1	1-bit capture indicator for *d_in*
result	Output	10	2's complement result
valid	Output	1	1-bit valid indication

Table 1: Module Port Signal and Function Table

As shown by Table 1, the module takes in 2's complement values for the arithmetic inputs and gives a 2's complement value for the output. As a result, the arithmetic inputs and outputs are signed values with the input, *d_in*, being able to express integers from -128 to 127 and the output, *result*, being able to express integers from -512 to 511. The reset signal is driven low at the start of any

simulation for this design in order to ensure that it starts at a known state. The 1-bit input, *capture*, ensures that the value of d_in should be stored in the appropriate register (A, B, C, D). The 1-bit output, *valid*, will become high once every register (A, B, C, D) is assigned a value. Once valid is high, it will remain high until a reset is driven low.

1.2 Top Level Module

The top-level module of this project, appropriately named *project2*, contains all of the inputs and outputs for the design. Figure 1 shows the source code of the module, *project2*. As shown in the figure, this module was written using datapath modeling due to project specifications. The primary purpose of this module is to drive the module *controller* and module *datapath* with their appropriate inputs and outputs. In addition,*project2* creates a connection between the two modules through the use of the 4-bit bus wire *mux_port*. This is used to indicate to the datapath module if a register (A, B, C, or D) should be assigned the value of d_in. As it drives a 2:1 multiplexer for the datapath module, it is called *mux_port*.

```
module project2(
    input reset_n,
    input clock,
    input [7:0] d_in,
    input [1:0] op,
    input capture,
    output [9:0] result,
    output valid
    );
    wire [3:0] mux_port;
    wire [7:0] reg8A;
    wire [7:0] reg8B;
    wire [7:0] reg8C;
    wire [7:0] reg8D;

    controller u1(.capture(capture),.op(op),
    .reset_n(reset_n),.clock(clock),
    .valid(valid),.mux_in(mux_port));
    datapath u2(.d_in(d_in), .clock(clock),
    .reset_n(reset_n), .mux_in(mux_port),
    .reg8A(reg8A), .reg8B(reg8B),.reg8C(reg8C),
    .reg8D(reg8D), .result(result));
endmodule
```

Figure 1: Source code of module, *project2*

The other 4 wires (*reg8A*, *reg8B*, *reg8C*, *reg8D*) contain the value of registers A, B, C, and D. Since they are only used in the datapath module, they would not need to be included in the top-level module. However, they were included in accordance with the automatic grader for this project.

1.3 Controller Module

The controller module of this project, appropriately named *controller*, contains all of the logic used for control and input validation. Figure 2 shows the source code of the module. As shown in the figure, this module was written using datapath modeling due to project specifications. The module utilizes a 2 by 4 decoder and 4 AND gates to drive the inter-module 4-bit bus, *mux_in*. The 2 by 4 decoder drives a single bit on the 4-bit bus, *y*, high, with the LSB corresponding to register A and the MSB corresponding to register D. The 4-bit bus, *y*, will only drive the appropriate bit of *mux_in* high when the input, *capture*, is driven high. This behavior was done through the use of an AND gate. This module also contains the logic for the 1-bit output, *valid* (stored in a register), and drives it high when the arithmetic output, *result*, is finalized. This portion of the module utilizes a custom-designed sequential latching mechanism that captures a high signal and maintains that state until the 1-bit input, *reset_n*, transitions to a low state, effectively resetting the latch. This behavior was implemented using a combination of D Flip-Flops and combinational logic.

```
module controller(
input capture,
input [1:0] op,
input reset_n,
input clock,
output valid,
output [3:0] mux_in
);
wire [3:0] y;
wire [3:0] or_in;
wire [3:0] q;
wire pre_val;
decoder u1(.sel(op), .y(y));
assign mux_in[0]=y[0]&capture;
assign mux_in[1]=y[1]&capture;
assign mux_in[2]=y[2]&capture;
assign mux_in[3]=y[3]&capture;

assign or_in[0] = (reset_n&q[0])|(mux_in[0]);
assign or_in[1] = (reset_n&q[1])|(mux_in[1]);
assign or_in[2] = (reset_n&q[2])|(mux_in[2]);
assign or_in[3] = (reset_n&q[3])|(mux_in[3]);
dff u2(clock,or_in[0],q[0]);
dff u3(clock,or_in[1],q[1]);
dff u4(clock,or_in[2],q[2]);
dff u5(clock,or_in[3],q[3]);
assign pre_val=reset_n&q[0]&q[1]&q[2]&q[3];
dff u6(clock,pre_val,valid);

endmodule
```

Figure 2: Source code of module, *controller*

1.4 Datapath Module

The datapath module of this project, appropriately named *datapath*, is responsible for the movement, storage, and arithmetic operations of the input operands. Figure 3 shows the source code of the module. As shown in the figure, this module was written using datapath modeling due to project specifications. The datapath module contains the registers for A, B, C, and D. It assigns the value of *d_in* to the appropriate register depending on the input, *mux_in*, which is driven by the module, *controller*. The datapath module will choose to either recycle the old input kept in registers A-D or assign it the new value of *d_in* through the use of a 2 to 1 8-bit multiplexer module. The multiplexer's selection is controlled by *mux_in*. For readability of the source code, buses of nets are used for assigning values to registers. The remaining components of the datapath module are used to perform the calculation (A-B)+(C-D). The result of this calculation is continuously assigned to the 10-bit output port, *result*. This approach is valid due to the 1-bit output, *valid*, which turns high only once a valid output is reached. Consequently, the state of the *result* remains inconsequential until all input operands, A through D, have been given. It is important to note that all registers in this module will be cleared if *reset_n* is driven low. This behavior is performed through the use of AND Gates on the input for all registers. The *reset_n* signal is sign extended to the full length of the register to ensure all bits of the registers are cleared. Regarding the implementation of sign extension, inputs to the 10-bit ripple carry adder module, *rca_10bit*, were extended to prevent issues with computation for the 9th and 10th bits.

```
module datapath(
input [7:0] d_in,
input clock,
input reset_n,
input [3:0] mux_in,
output [7:0] reg8A,
output [7:0] reg8B,
output [7:0] reg8C,
output [7:0] reg8D,
output [9:0] result
);

    wire [7:0] pre_A;
    wire [7:0] pre_B;
    wire [7:0] pre_C;
    wire [7:0] pre_D;
    wire Cout_1;
    wire Cout_2;
    wire Cout_3;
    wire [9:0] Sum_1;
    wire [9:0] Sum_2;
    wire [9:0] pre_result;
    mux_2to1 u1(.sel(mux_in[0]), .i0(reg8A), .i1(d_in), .out(pre_A));
    mux_2to1 u2(.sel(mux_in[1]), .i0(reg8B), .i1(d_in), .out(pre_B));
    mux_2to1 u3(.sel(mux_in[2]), .i0(reg8C), .i1(d_in), .out(pre_C));
    mux_2to1 u4(.sel(mux_in[3]), .i0(reg8D), .i1(d_in), .out(pre_D));
    reg8 u5(clock, ({8{reset_n}}&pre_A), reg8A);
    reg8 u6(clock, ({8{reset_n}}&pre_B), reg8B);
    reg8 u7(clock, ({8{reset_n}}&pre_C), reg8C);
    reg8 u8(clock, ({8{reset_n}}&pre_D), reg8D);
    rca_10bit u9({{2{reg8A[7]}},reg8A},~{{2{reg8B[7]}},reg8B},1'b1,Sum_1,Cout_1);
    rca_10bit u10({{2{reg8C[7]}},reg8C},~{{2{reg8D[7]}},reg8D},1'b1,Sum_2,Cout_2);
    rca_10bit u11(Sum_1,Sum_2,1'b0,pre_result[9:0],Cout_3);
    reg10 u13(.clock(clock),.d({10{reset_n}}&pre_result[9:0]),.q(result[9:0]));

endmodule
```

Figure 3: Source code of module, *datapath*

1.5 Other Modules

The *controller* and *datapath* modules make use of other modules that help to manipulate, store, and select data. This was done to make this design more portable, easier to read, and easier to debug. Table 2 contains a list of modules used in this design, aside from *controller*, *datapath*, and *project2*. The module, *full_adder*, is not used in either the *datapath* or *controller* modules, as shown by Table 2; rather, the module is used for *rca_10bit* as the structure of a ripple carry adder utilizes full adders.

Module Name	**Function**	**Datapath**	**Controller**
decoder	Decodes a 2 bit opcodes to drive one of four outputs high		✓
dff	Basic 1-bit D Flip-Flop memory element	✓	✓
mux_2to1	Selects between 2 8-bit values	✓	
rca_10bit	Performs 10-bit addition with 1 bit carry in	✓	
full_adder	Single-bit addition with 1 bit carry in		
reg10	10-bit register synced with the clock	✓	
reg8	8-bit register synced with the clock	✓	

Table 2: Other modules used in the design

The sizes of inputs and outputs for certain modules were chosen specifically to simplify the structure and design of the *datapath* module. This is demonstrated by the module, *rca_10bit*. As can be inferred from the name, this module performs 10-bit addition. This module has two 10-bit inputs and a 10-bit output. It was implemented in this manner as the final computation for the design is 10 bits in length (output *result*) and allows for the datapath module to only need to instantiate *rca_10bit* three times:

A-B, C-D, and the sum of the result of A-B and C-D. If the ripple carry adder module was created for 8-bit numbers, then additional *full_adder* modules would need to be included in the datapath module. This would complicate the code for both readability and debugging.

The sizes for the inputs and outputs of the register modules, *reg8* and *reg10*, were also chosen to simplify the structure and design of the *datapath* module. While 8 D Flip-Flops could have been used for storing values, it would have greatly complicated the structure of the datapath module. Instead, as shown in figure 4, these D Flip-Flops are contained in *reg8*. The internal design of *reg10* makes use of *reg8* by containing an instance of it, along with 2 D Flip-Flops to create a 10-bit register. This is shown in Figure 5.

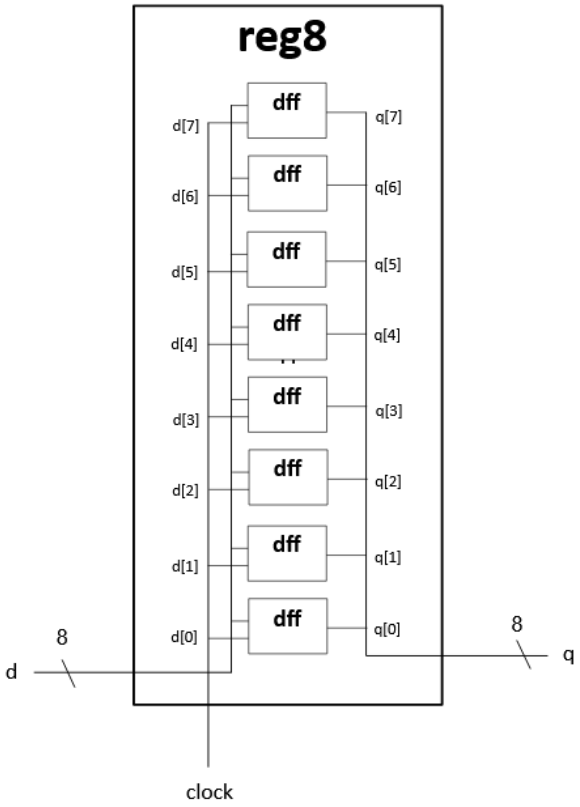

Figure 4: Diagram of the module, *reg8*

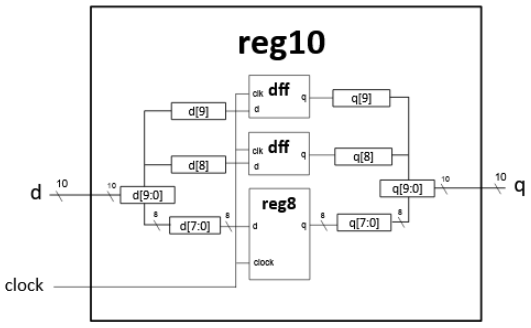

Figure 5: Diagram of the module, *reg10*

2 Simulation, Results and Analysis

To verify the correctness of this project, four sets of test values were chosen. These sets assign various values to the inputs of this design, aside from *clock*, which will follow the same pattern for every set of test values, as the standard cycle for this design will always have the clock be a constant period. Each test case will always have the reset activated before a set of inputs is entered.

2.1 Test Case 1: Ideal Operation

op (2-bit)	d_in (8-bit)	capture (1-bit)	Operand
2'b00	8'b0010 0101	1'b1	A (37)
2'b01	8'b0000 1010	1'b1	B (10)
2'b10	8'b0011 1110	1'b1	C (62)
2'b11	8'b0000 0100	1'b1	D (4)

Table 3: Ideal Operation Test Vectors

This first set of test values contains the ideal operation for the design. The values for the selection bit, *op*, were given in increasing order from 0 to 3. This results in register A being assigned a value, then register B, then register C, then register D. The values of *d_in* are all positive numbers, where A>B and C>D, resulting in the 10-bit output *result* being a positive number as well. The value of *d_in* is always captured as indicated by the 1-bit input, *capture*, being high for all of the test values. This was done to ensure that all operand values were to be utilized in the computation for this ideal scenario.

2.2 Test Case 2: Negative Operation

op (2-bit)	d_in (8-bit)	capture (1-bit)	Operand
2'b00	8'b1110 1011	1'b1	A (-21)
2'b01	8'b1111 1100	1'b1	B (-4)
2'b10	8'b1100 0001	1'b1	C (-63)
2'b11	8'b1111 1000	1'b1	D (-8)

Table 4: Negative Operation Test Vectors

The second test set evaluates the arithmetic units, specifically focusing on the handling of two's complement signed arithmetic. As shown in Table 4, by the MSB of all 4 values of *d_in* being equal to 1, all inputs are negative. The inputs for this unit were chosen partly because they will result in a negative 10-bit output. This was done as it is critical to verify that the datapath correctly processes negative values during the subtraction phases. By selecting operands where A<B and C<D, the system's ability to perform sign extension and 10-bit signed addition is tested.

2.3 Test Case 3: Capture Logic and Operation

op (2-bit)	d_in (8-bit)	capture (1-bit)	Operand
2'b01	8'b0100 1000	1'b1	B (72)
2'b00	8'b0000 1010	1'b1	A (10)
2'b11	8'b0111 1010	1'b0	D (122)
2'b01	8'b0000 1001	1'b1	B (9)
2'b10	8'b0011 1100	1'b1	C (60)
2'b11	8'b0111 1101	1'b1	D (125)

Table 5: Capture Logic and Operation Test Vectors

This test case uses several values to simulate several non-ideal behaviors for the capture logic. The first behavior is "writing over" a previously entered value for a register. For this test case, it is done by having the value 8'b1000 1000 be assigned to register B (as shown through op = 2'b01), then, after other values are given, the value 8'b0000 1001 is assigned to register B. The second non-ideal behavior for the capture logic is having inputs given that shouldn't be captured. This is done by assigning the value 8'b0111 1010 to register D when capture = 1'b0 and then having the value 8'b0111 1101 assigned to register D when capture = 1'b1. The third behavior is having inputs for the registers A-D being declared out of order. This is done by having the values for *op* being entered not in increasing order. This test case also verifies the behavior of the valid bit, as inputs are overwritten and entered out of order.

2.4 Test Case 4: Maximum Edge Case

op (2-bit)	**d_in (8-bit)**	**capture (1-bit)**	**Operand**
2'b00	8'b0111 1111	1'b1	A (127)
2'b01	8'b1000 0000	1'b1	B (-128)
2'b10	8'b0111 1111	1'b1	C (127)
2'b11	8'b1000 0000	1'b1	D (-128)

Table 6: Maximum Edge Case Test Vectors

The fourth test set evaluates the design with the maximum value of the 10-bit result. This is done by providing the largest possible positive values for A and C and the largest possible negative values for B and D. Since the inputs are all 8-bit signed 2's complement, they have a range of -128 to 127. This verifies that the 10-bit output bus correctly accommodates the sum without truncation or overflow, confirming the success of the ripple carry adders used in the dataflow module. Not shown in the above table is that after these values are entered and the outputs are given, *reset_n* will be driven low for a second period in order to test reset functionality.

2.5 Testbench designing

Four testbenches are created and executed to verify the success of this design. Each testbench executes a test case and provides the 2 outputs of the design, *project2*. These outputs, *result* and *valid*, are compared to the expected values for each test case. To ensure the simulation accurately reflects the intended operation of the unit, all tests are conducted under the following conditions.

- The time unit of each simulation will be 1 ns.
- The period of the 1-bit input *clock* will be 10 ns.
- Before the start of each test case, *reset_n* will be set high for 6 ns then remain low for the rest of that test case's simulation.
- The delay between inputs will be 10 ns.
- Inputs for each test case are strictly limited to the defined test case vectors.

These conditions will be maintained to ensure uniformity for each test case and were made in accordance with project specifications. These conditionals also allow for consistent timing among simulations. As mentioned in section 2.4, the testbench for the fourth test case will have a second change in the input, *reset_n*. The input will be driven low for a duration of 10ns, which will occur 5ns after the display of the output. As each testbench's structure is similar, the source code for Test Case 1 is representative of each case and is shown in Figure 6. As shown in the figure, the testbench follows each condition to ensure the simulation environment remains consistent across all test cases. The instance of *project2* used in the testbench is called *dut*, as that is the standard naming convention in Verilog for the module under testing in a testbench. In addition to generating waveforms, the testbench uses *$display* statements to output the signal values to the TCL Console in decimal format. As shown in Figure 6, these values are displayed 25ns after the last input is given. This delay is used

to ensure that the outputs are recorded after the design is finished with computation and has enough time for values to be sent through registers.

```
module project2_tb_case1;
    reg reset_n;
    reg clock;
    reg [7:0] d_in;
    reg [1:0] op;
    reg capture;
    wire [9:0] result;
    wire valid;
    Project2 dut(reset_n, clock, d_in, op, capture, result, valid);
    initial clock = 0;
    always #5 begin clock = ~clock; end

    initial begin
    reset_n = 1'b0;
    #10 reset_n = 1'b1;
        d_in=8'sd37;
        capture=1;
        op=2'b00;
    #10 d_in=8'sd10;
        capture=1;
        op=2'b01;
    #10 d_in=8'sd62;
        capture=1;
        op=2'b10;
    #10 d_in=8'sd4;
        capture=1;
        op=2'b11;
    #25 $display("Test Case 1: Valid = %d, Result = %d", valid,result);
    $finish;
    end
endmodule
```

Figure 6: Source Code of the Testbench for Test Case 1

2.6 Simulation Results

After every test case was executed, the calculated outputs were gathered. As each value of the actual result matches the expected result for each test case, the design is deemed successful.

As mentioned previously, the fourth test case was used to test the maximum edge case for the ripple carry adders and a 10-bit result. The computation being done for this design is (A-B)+(C-D), where A-D are 8-bit 2's complement numbers. The range of an 8-bit 2's complement number is -128 to 127. Given this range, the highest possible result happens when A and C are both 127, and B and D are both -128, resulting in a maximum value of 510. The minimum result happens when A and C are both -128, and B and D are both 127, resulting in a value of -510. The 10-bit 2's complement output, *result*, has a range of -512 to 511. Therefore, it can be concluded that this design can handle any value (8-bit 2's complement) for its inputs without overflow. This attests to the robustness of the design.

Figures 7 to 10 provide an alternative view of the input and output of the design for each of the 4 test cases. Each figure displays the inputs and outputs for each test case through the use of AMD Vivado's waveform output. By utilizing this type of view, the timing for each test can be analyzed to ensure that the design meets the required specifications. For the input *d_in* and the output *result*, their values are displayed as signed decimals. While hexadecimal or binary could have been used, the sign of arithmetic inputs and outputs is shown clearest by signed decimal.

Figure 7 shows the waveforms output for the first test case, designed to show the design under ideal operations. As shown in the figure, the 2-bit input *op* increases sequentially from 0 to 3. This results in register A being assigned to first, followed by register B, then register C, and finally register D. Every input of *d_in* is positive in order to test the summation properties of the arithmetic components of the design. At 55 ns into the simulation, the 1-bit output, *valid*, transitions to 1'b1. This indicates that the result of the design for the given inputs is 10'd85. This matches the expected output for the given inputs.

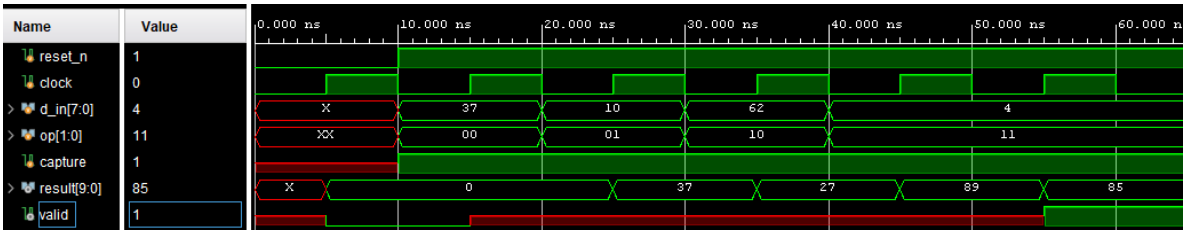

Figure 7: Full Waveforms Output of Testbench for Test Case 1

Figure 8 shows the waveforms output for the second test case, designed to show the design under negative operands. As shown in the figure, the 2-bit input *op* increases sequentially from 0 to 3. This, as with test case 1, results in register A being assigned to first, followed by register B, then register C, and finally register D. Every input of *d_in* is negative in order to test the subtractive properties of the arithmetic components of the design. At 55 ns into the simulation, the 1-bit output, *valid*, transitions to 1'b1. This indicates that the result of the design for the given inputs is -10'sd72. This matches the expected output for the given inputs.

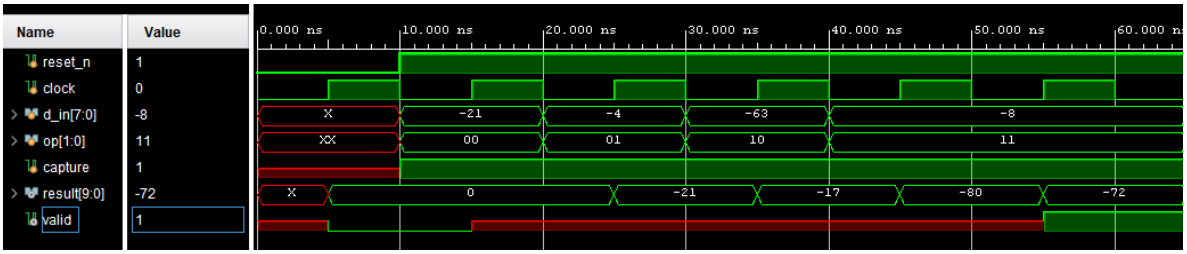

Figure 8: Full Waveforms Output of Testbench for Test Case 2

Figure 9 shows the waveforms output for the third test case, designed to test the design's capture logic and capture operations. As shown in the figure, the 2-bit input *op* doesn't increase sequentially from 0 to 3, with 2'b01 being the first input for *op*. This results in register A not being assigned first, and instead, register B. The inputs of *d_in* fluctuate between being negative and positive in order to test the summation properties of the arithmetic components of the design under both types of numbers. At 65 ns into the simulation, the 1-bit output, *valid*, transitions to 1'b1. This indicates that the result of the design for the given inputs is -10'sd64. This matches the expected output for the given inputs and confirms that the capture logic and capture operations are successful.

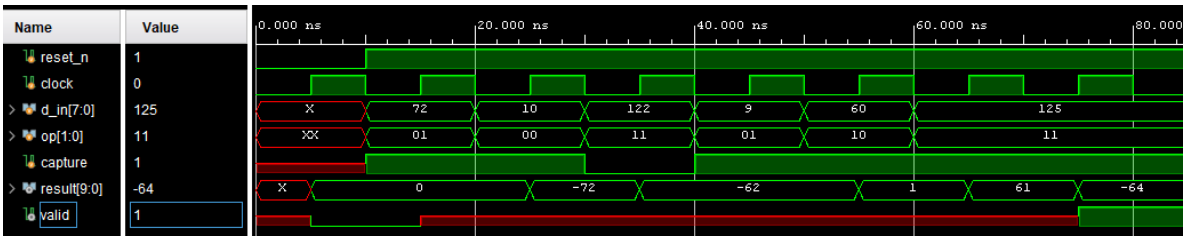

Figure 9: Full Waveforms Output of Testbench for Test Case 3

Figure 10 shows the waveforms output for the fourth test case, designed to show the design with maximum values for the operands to simulate a critical edge case. As shown in the figure, the 2-bit input *op* increases sequentially from 0 to 3. This results in register A being assigned to first, followed by register B, then register C, and finally register D. The input of *d_in* when *op* is 2'b00 (A) and 2'b10 (C) is 127, while for 2'b01 (B) and 2'b11 (D) is -128. This stems from the input, *d_in* having a maximum value of 127 and a minimum value of -128. As the calculation being performed by the design is A-B+C-D, A and C need to be the largest possible values, while B and D need to be the smallest possible values in order to have the output, *result* be a maximum. At 55 ns into the simulation, the

1-bit output, *valid*, transitions to 1'b1. This indicates that the result of the design for the given inputs is 10'd510. This matches the expected output for the given inputs.

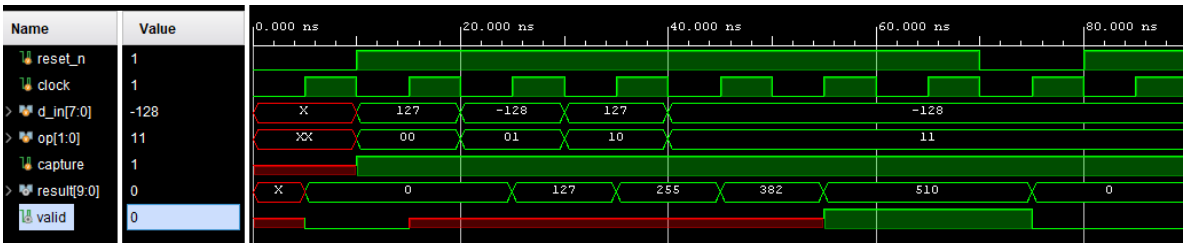

Figure 10: Full Waveforms Output of Testbench for Test Case 4

In addition to an edge case, the reset functionality of the design is tested through test case 4. This is done by having *reset_n* be set low after the result of the design is given. The expected result from this is that the outputs, *result* and *valid*, will be set to 0 once the next positive edge of the clock occurs, as the outputs are driven by registers that are updated on the positive edge of the clock. As shown through figure 10, this exact behavior occurs. From this, it can be concluded that the reset functionality of this design is successful and correctly performed. As test cases 1, 2 and 4 have only 4 inputs, they are 65 ns in length from the start of the simulation to the displaying of the outputs. Test case 3 is longer at 85 ns due to having two more inputs. As shown by the figures, the correct *result* value occurs at the same point that valid becomes high, verifying a key requirement for this design.

From 15 ns to 55 ns (for test cases 1, 2 and, 4) or 65 ns (for test case 3), the value of the 1-bit output, *valid* is X. This occurs because no value is provided to the input *op[1:0]* and *capture* at the start of the simulation. This can be solved by driving the inputs for the entirety of the simulation. The reason why no values were provided for the inputs at the start was that *reset_n* needs to be driven low before inputs for the test case are entered into the simulation. Adding a reset signal directly to the D Flip-Flop instances for the controller module would also solve the issue. This behavior is inconsequential to declaring the success of the design, as the requirement for a valid signal is that it needs to be asserted once the result of the design is available.

Figure 11 provides an expanded view of the output waveform between 0 ns and 12 ns. This outcome is shared across the waveforms for each of the test cases. The figure shows the value of every input, aside from *clock* and *reset_n*, being X. This is also shown on Figures 7 - 10, though not as clearly. As shown in 11, all of the inputs that have a value of X change to a known value at 10 ns. This occurs because from 0 ns to 10 ns, *clock* and *reset_n* are the only inputs being driven by the testbench. It is not until 10 ns that the other inputs are first defined in the testbench. The reason why the other inputs are not defined at 0 ns and instead at 10 ns is that a reset needs to occur before the design can take in the first value for the other inputs.

Figure 11 also shows the value of outputs *result* and *valid*, being X from 0 to 5 ns. The outputs remain X until 5 ns, where they reflect their expected values. This occurs due to the design of the system. Outputs *result* and *valid* have registers that hold their values. When a register is first initialized in AMD Vivado, it has a value of X, which is why the outputs begin the simulation with a value of X. The registers for the design use D Flip-Flops to assign new values, and D Flip-Flops are updated on the positive edge of the clock signal. As the clock signal's positive edge occurs at 5 ns into the simulation, from 0 ns to 5 ns, the value of the registers that drive the outputs is unknown. In order to have the outputs be defined for the entirety of the simulation, the clock signal would need to begin high, not low. The structure of the D Flip-Flops follows the design given by the project specifications. This initial delay is an inherent characteristic of D Flip-Flop logic rather than a functional error. From this analysis, it can be concluded that the initial unknown state of the inputs and the outputs is expected behavior rather than a result of a functional error in the design.

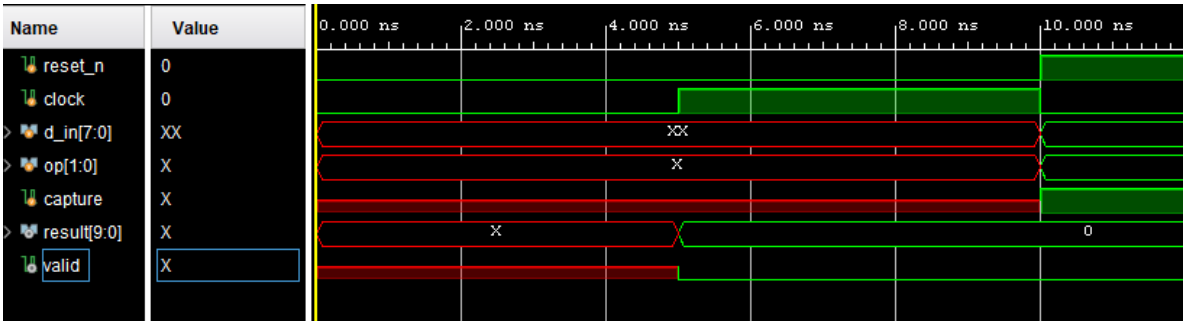

Figure 11: Enhanced Waveform of the first testbench from 0 ns to 12 ns

A potential limitation in the verification process is that only a small number of test cases were utilized. For the 2's complement 8-bit input, *d_in*, there are $2^8 = 256$ possible combinations. When including the 2-bit *op* input and the 1-bit input, *capture*, there are $2^3 \times 2^8 = 2048$ total possible input values per clock cycle. While the simulation scenarios cover the primary operational modes, they represent only a fraction of the possible input sequences over time. However, it is impractical to simulate every possible sequence due to the time required for manual verification and documentation. Therefore, representative test cases were selected to confirm the success of the design under a variety of conditions.

As the outputs from the TCL console and waveforms for each of the 4 test cases match the expected behavior for the design, the Verilog design and its logical structure are deemed successful.

3 Conclusions

A correct implementation and verification of a complex design that stores inputs and performs arithmetic operations is shown in this report. Incorporating selection, reset, and capture functionally while also implementing an output indicator is also shown in this report. This was accomplished through mostly dataflow modeling to define the proper behavior for a controller and datapath module. Several of Verilog's abilities, such as modules, instances, and register were also demonstrated in this report. While the design was functionally successful, an area for future improvement involves timing and resource optimization. Since the test cases focused solely on functional correctness, separate timing analysis could be conducted to determine how the state register performance impacts the emulated hardware of the system. These efficiency metrics could be incorporated into future projects when selecting test cases. In addition, the examination and impact of X and Z inputs could have also been examined. Nonetheless, the capabilities of Verilog were demonstrated to implement a complex design through various requirements and specifications. Rather than requiring physical components like logic gates, wires, and chips, or expensive hardware like an FPGA, the design was fully verified digitally with minimal resources used.

This report was compiled using LaTeX .